

SELECT * FROM binary |

Vibe Reversing Across IDA, Ghidra, and Binary Ninja

Elias Bachaalany (@eliasbchlny / @allthingsida)

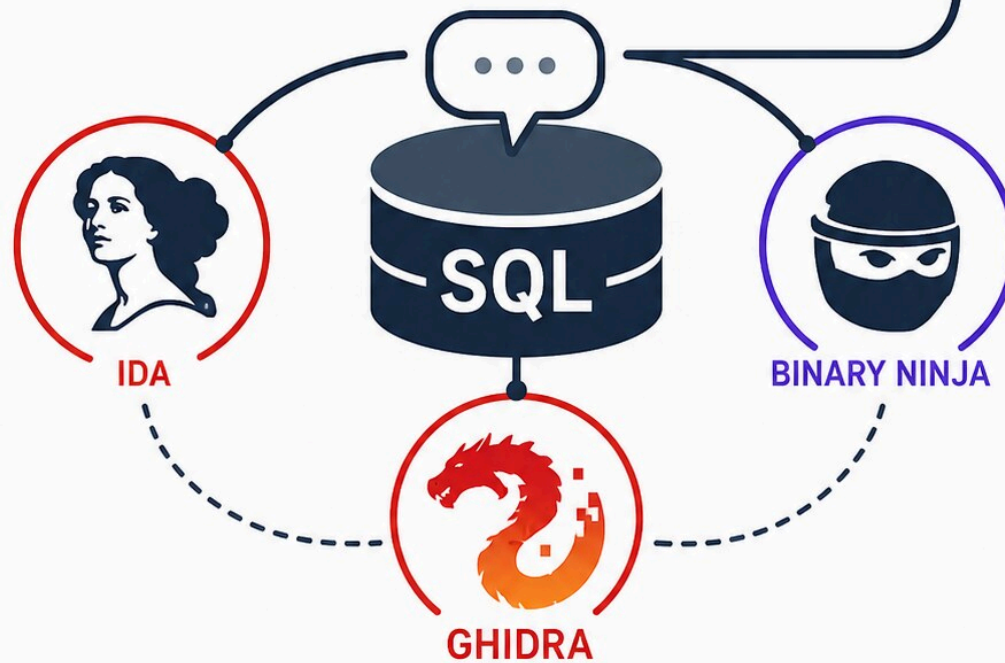
REcon 2026 · Montreal

`idasql` (IDA Pro) · `ghidrasql` (Ghidra) · `bnsql` (Binary Ninja)

+ `idasql-skills` · `ghidrasql-skills` · `bnsql-skills`

```
>_ SELECT *  
FROM binary;
```

VIBE REVERSING ACROSS
IDA, GHIDRA, AND BINARY NINJA



ONE LANGUAGE,
THREE TOOLS



READ-WRITE
ANALYSIS



CROSS-TOOL
ANNOTATIONS



AI-DRIVEN
REVERSE ENGINEERING

Vibe coding

Describe the code you want. The agent writes it.

Vibe reversing

Describe what you want to know about a binary. The agent finds it.

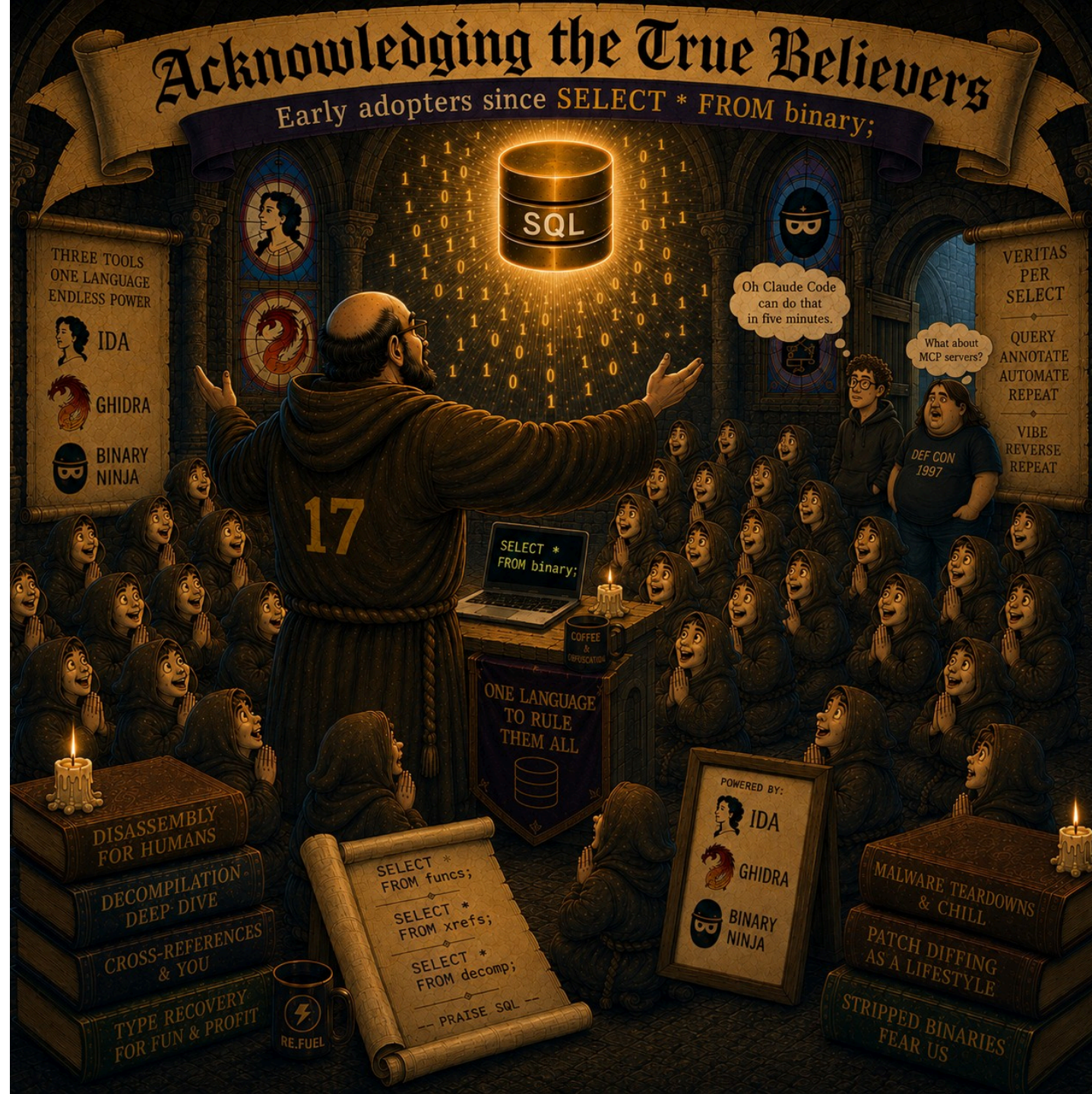
AGENDA

Where we're going

1. **Motivation** - why not just bolt tools onto an agent
2. **libxsql** - the foundation, and how it works inside
3. **The xSQL family** - idasql · bnsql · ghidrasql · r2sql
4. **A closer look: idasql** - tables, views, functions, abilities
5. **libghidra & ghidrasql** - bridging a Java engine
6. **AI integration** - skills, and letting the agent drive
7. **Quick start** - install, headless vs. assisted
8. **Vibe Reversing** - live

Acknowledging the True Believers

Early adopters since `SELECT * FROM binary;`



Hail the true believers

I preached the gospel of `SELECT * FROM binary` to anyone who'd listen. These kind souls heard me out, kicked the tires, and answered the call:

- @rootbsd
- Alex Matrosov
- @Tzion0
- Peter Goodman
- @hasherezade
- Igor Skochinsky
- @bivex
- Rodrigo Branco
- Antonio Nappa
- @EZForever
- Stephen Sims (@Steph3nSims)
- Conor Quigley
- @Cr4sh
- Ian Norris
- @singleghost2
- Jordan Wiens
- @Oxygen1a1
- Arnaud Diederer
- @torsts
- Andreas Klopsch
- @heheda123123
- Chris Hernandez
- Arnold Osipov
- Paolo Montesel
- [Ciarán McNally](#)

No endorsement implied. Acknowledged with gratitude for their feedback, their kindness and generosity, and for being early adopters. 🙏

PART 1

Motivation

How do you give an AI agent access to a reversing tool?

Every RE tool has its own API

Tool	How you script it
IDA Pro	IDAPython · IDC · C++ SDK
Ghidra	Java · Jython · PyGhidra
Binary Ninja	C++ SDK · Python API
radare2	r2pipe · commands

Each is powerful - and **mutually incompatible**.

- a human who switches tools **re-learns everything**
- an agent can't drive any of them **without tool-specific glue**

The tempting answer: MCP

99% of people reach for **MCP** - one tool call per operation:

```
list_functions()      get_strings()      get_xrefs_to()
rename_function()    set_comment()      get_decompilation()
apply_type()         add_bookmark()     set_variable_name() ...
```

It's the 2023 function-calling idea, formalized. It's what the industry leans on.

Besides - where's the fun in doing what everyone already does?

Why one-tool-per-operation hurts

- **Context bloat** - every tool ships a JSON schema on **every request**; an IDA MCP can load **100+ tool definitions** before you ask anything. That budget goes to the tool list, not the binary.
- **Fixed tools don't compose** - each answers exactly what its author anticipated. A compound question - *"which functions decrypt a string and then call a network API?"* - means stitching a dozen calls by hand, or dropping to a script. In SQL it's one `JOIN`.
- **Re-built for every engine** - IDA, Ghidra, and Binary Ninja each need their own hundred-tool surface to build, document, and keep in sync.

New question ⇒ new tool ⇒ patch the server, ship again.

I've built this before

- the "binary as a database" idea is old - I had it ~**2022–2023**, but there were no coding agents then to execute it at scale, so it waited
- in **2023** I built `ask_ida` : a full agentic RE harness - tool-call loop, slash commands, prompts, meta-commands - **before** CLI agents, **before** MCP
- I tried both ends: a wall of tools, then - by its last iteration - a **single** run-IDAPython tool backed by a curated skills library. Neither felt right as the *interface*.

The lesson: a better interface - not more tools, and not one catch-all code tool.

So flip it

What if the interface were a language that is...

- **composable** - not a fixed list of operations
- **already known** by every LLM, large or small
- able to **unify every RE tool at once**

SELECT * FROM binary

PART 2

libxsql

The foundation - and how it actually works

What is **libxsql**?

A small C++ library that turns **any tool's internals** into **live SQLite virtual tables**.

Two principles:

- **Live** - no export, no intermediate dump. The table *is* the engine's state.
- **No caching** - caching, if any, lives only for the query's lifetime.

`libxsql` is the prerequisite for the whole xSQL family.

The binary is a database.

Not a dump.

Not an export.

A live view over the analysis engine – that you can **read and write**.

The ordinary question, as a query

"What are the biggest functions?"

```
# the script you'd write today
import idutils, ida_funcs
rows = [(ida_funcs.get_func_name(ea), ida_funcs.get_func(ea).size)
        for ea in idutils.Functions()]
rows.sort(key=lambda r: r[1], reverse=True)
```

```
-- the same question, as a question
SELECT name, size FROM funcs ORDER BY size DESC LIMIT 10;
```

Functions are rows. You describe the result.

The rest of triage is rows too

```
-- what words does it contain?  
SELECT content FROM strings WHERE content LIKE 'op_%';  
  
-- what APIs does it use?  
SELECT module, name FROM imports ORDER BY module, name;  
  
-- who points at this function?  
SELECT from_func FROM xrefs WHERE to_ea = 0x140001BD0;
```

funcs · strings · imports · xrefs - same shape, every time.

And the rows write back

```
UPDATE funcs
SET name = 'exec_command',
    prototype = 'void exec_command(command_t *cmd)'
WHERE address = 0x140001BD0;

INSERT INTO bookmarks(address, description)
VALUES (0x1400016A0, 'unchecked file-open result');
```

- changes land in the tool **immediately**
- a `prototype` update can reshape the decompiler output
- the RE tool stays the source of truth

How does libxsql actually do this?

A table is a **declarative builder** - a few lambdas mapping data → columns.

No subclassing, no boilerplate:

```
auto def = xsql::table("products")
    .count([&]{ return products.size(); })
    .column_int ("id",    [&](size_t i){ return products[i].id;  })
    .column_text("name",  [&](size_t i){ return products[i].name; })
    .build();

db.register_and_create_table(def);    // now: SELECT * FROM products
```

The `CREATE TABLE` schema is generated from the columns for you.

Now make it live over IDA

Swap the lambdas for **IDA SDK calls** - that's the whole trick:

```
auto def = xsql::table("funcs")
    .count([&]{ return get_func_qty(); })           // SDK: how many
    .column_text("name", [&](size_t i){            // SDK: the name
        return get_func_name(getn_func(i)->start_ea); })
    .column_int64("size", [&](size_t i){           // SDK: func_t::size()
        return getn_func(i)->size(); })
    .build();
```

No export. No cache. A live view over the IDB.

Making a column writable

Add a setter - read lambda + write lambda:

```
.column_text_rw("name",  
    [&](const FuncRow& r){ return get_func_name(r.start_ea); },    // read  
    [&](FuncRow& r, const char* n){  
        return set_name(r.start_ea, n, SN_CHECK) != 0; })        // write
```

```
UPDATE funcs SET name='exec_command' WHERE address=0x140001BD0;
```

→ calls `set_name(...)` - the rename lands live in IDA.

Same pattern: `prototype` → `apply_cdecl`, `comment` → `set_func_cmt`,
`row DELETE` → `del_func`, `INSERT` → `add_func`.

Why this beats a wall of tools

- every column = **one lambda** over a tool call
- a new capability is a **new column**, not the 117th tool
- the agent gets `SELECT` / `INSERT` / `UPDATE` / `DELETE` over the **whole surface** for free

Power comes from composition - not a fixed list of operations.

PART 3

The xSQL family

One idea, every top RE engine

Meet the family

Tool	Engine	Runs as
idasql	IDA Pro	CLI + plugin
ghidrasql	Ghidra	CLI + extension
bnsql	Binary Ninja	CLI + plugin
r2sql	radare2	CLI + plugin

All sit on the same idea: `libxsql` virtual tables over a **live** RE engine.

More of the family: `pdbsql` · `dwarfsql` · `clangsql` - same SQL, different sources.

One query, three engines

```
SELECT name, size FROM funcs ORDER BY size DESC LIMIT 10;
```

	IDA	Ghidra	Binary Ninja
functions	243	210	312
main size	249	247	248

Same bytes. Same question. Three readings.

The differences are part of the analysis

What differs	Why it matters
function count (243/210/312)	boundary recovery, thunks, library classification
top function	each engine groups code differently
size by a byte or two	where a function <i>ends</i> differs
names	sub_* vs FUN_* vs tool-specific recovery

Port the question, then compare the answers.

PART 4

A closer look: idasql

Same methodology builds the whole family - we go deep on one

Where idasql comes from

- ~2022–2023 - the idea: expose a binary as live virtual tables. No coding agents yet to execute it at scale - so it waited.
- Q2 2023 - `ask_ida` (private): a full agentic RE harness - tool-call loop, slash commands, prompts, meta-commands. On and off, then **heavy refactoring through 2024** as CLI coding agents emerged. Rewritten ~7 times.
- Winter 2025 - those rewrites led to a new approach: **idasql**, a SQL-first surface rewritten out of `ask_ida`; `ask_ida` decommissioned.
- v0.0.10 - even idasql's *own* built-in agent was removed → fully **agent-agnostic**: any agent drives it through SQL.
- 2026 - the family follows on the same `libxsql` idea: bnsql, ghidrasql + libghidra, r2sql. Today: **idasql v0.0.17**.

The idea was always there - coding agents are what let it become this complete.

Why not build my own harness?

`ask_ida` was a harness. So why not keep building one?

- a harness is a **whole product** - a business division of its own
- and **you can't please everyone** - harness taste is personal (I like my own, and I prefer CLI)
- I'd rather focus on the **core tools**: the SQL surface
- great harnesses already exist. Pick any:

Claude Code · Codex (CLI/App) · Copilot (CLI/App) · Pi Agent · Cursor · OMP · OpenCode

Harnesses are separate products - bring your own. If a tool bundles one, it should be open and well-maintained.

The surface, at a glance

30+ tables · named views · scalar functions over one IDB.

Layer	What it is	Example
Tables	nouns - the engine's state as rows	<code>funcs</code> , <code>xrefs</code> , <code>types</code>
Views	analysis, precomputed (joins)	<code>callers</code> , <code>string_refs</code>
Functions	verbs + the escape hatch	<code>decompile(ea)</code> , <code>parse_decls(...)</code>

Learn the model once; it covers the whole engine.

The table catalog

Group	Tables
code	<code>funcs</code> · <code>instructions</code> · <code>blocks</code> · <code>segments</code> · <code>heads</code>
xrefs	<code>xrefs</code> · <code>callers</code> · <code>callees</code> · <code>data_refs</code>
data	<code>strings</code> · <code>bytes</code> · <code>names</code> · <code>imports</code> · <code>entries</code>
types	<code>types</code> · <code>types_members</code> · <code>types_enum_values</code>
decompiler	<code>pseudocode</code> · <code>ctree</code> · <code>ctree_lvars</code> · <code>ctree_labels</code>
annotate	<code>comments</code> · <code>bookmarks</code> · <code>applied_types</code>
storage	<code>netnode_kv</code> · <code>dirtree_folders</code>

Views = analysis, precomputed

The hard joins ship as named views:

```
-- who calls this function, with names already resolved
SELECT caller_name, caller_addr FROM callers WHERE func_addr = 0x140001BD0;

-- strings + xrefs + containing function, joined for you
SELECT func_name, string_value FROM string_refs WHERE string_value LIKE '%file%';

-- indirect calls - a security-relevant pattern, as a view
SELECT call_ea, target_var_name FROM ctree_v_indirect_calls WHERE func_addr = 0x140001BD0;
```

callers · callees · string_refs · disasm_v_call_chains · ctree_v_* ·
types_v_inheritance

Scalar functions = verbs & escape hatch

```
SELECT decompile(0x140001BD0);      -- Hex-Rays pseudocode as text
SELECT disasm_func(0x140001BD0);    -- full function listing
SELECT gen_cfg_dot(0x140001BD0);    -- control-flow graph as DOT
SELECT parse_decls('typedef struct {...} command_t;'); -- import C types
SELECT save_database();             -- persist edits to the IDB
```

When a row isn't the right shape, a function is.

Ongoing work: re-engineering to minimize the escape hatches - favoring tables and views.

Triage in three queries

```
-- biggest functions
SELECT printf('0x%X', address) AS addr, name, size
FROM funcs ORDER BY size DESC LIMIT 10;

-- dependencies, by module
SELECT module, COUNT(*) AS n FROM imports GROUP BY module ORDER BY n DESC;

-- behavioral hints
SELECT printf('0x%X', address) AS addr, content
FROM strings WHERE length >= 8 ORDER BY length DESC LIMIT 40;
```

The first three questions of almost every session.

Scenario: xrefs, callers, call graph

```
-- callees of the dispatcher (named, via the view)
SELECT printf('0x%X', callee_addr) AS site, callee_name
FROM callees WHERE func_addr = 0x140001BD0;

-- who calls it
SELECT printf('0x%X', caller_addr) AS site, caller_name
FROM callers WHERE func_addr = 0x140001BD0;

-- hottest targets in the whole binary
SELECT to_ea, COUNT(*) AS callers FROM xrefs
WHERE is_code GROUP BY to_ea ORDER BY callers DESC LIMIT 10;

-- shortest path between two functions
SELECT step, func_name FROM shortest_path
WHERE from_addr = 0x1400016A0 AND to_addr = 0x140001BD0;
```

IDA-style caller/callee/call-graph analysis - in SQL.

Scenario: decompilation as data

```
-- the pseudocode, as text
SELECT decompile(0x140001BD0);

-- its local variables (name, type, is-it-an-argument)
SELECT idx, name, type, is_arg FROM ctree_lvars
WHERE func_addr = 0x140001BD0 ORDER BY idx;

-- calls that only run on some branches
SELECT ea, callee_name, branch FROM ctree_v_calls_in_ifs WHERE func_addr = 0x140001BD0;
```

Pseudocode, locals, and the AST - all queryable.

Annotate at every level

```
-- function: rename + retype + summarize, in one write
UPDATE funcs SET name='exec_command',
               prototype='void __fastcall exec_command(command_t *cmd);',
               rpt_comment='dispatches a command record'
WHERE address = 0x140001BD0;

-- decompiler local: rename + retype
UPDATE ctree_lvars SET name='cmd', type='command_t *'
WHERE func_addr = 0x140001BD0 AND idx = 0;

-- pseudocode comment, placed above the line
UPDATE pseudocode SET comment='tagged-union dispatch', comment_placement='block1'
WHERE func_addr = 0x140001BD0 AND ea = 0x140001BE8;
```

also: label renames · union-arm selection · per-operand number format · comments & bookmarks
(folded)

Recover types - live, in place

```
-- declare a struct + enum straight from C
SELECT parse_decls('
    typedef enum { op_empty=0, op_open=11, op_read=22 } operations_e;
    typedef struct { operations_e op; void *payload; } command_t;
');

-- apply a recovered type at an address
INSERT INTO applied_types(address, decl)
VALUES (0x140001BD0, 'void __fastcall exec_command(command_t *cmd);');

-- freeze the layout, then see free space to grow it
UPDATE types SET is_fixed = 1 WHERE name = 'command_t';
SELECT gap_offset, gap_size FROM type_gaps WHERE type_name = 'command_t';
```

Fixed-layout recovery - edit a struct in place, no rebuilds, no ordinal churn.

Bitfields & bitmask enums

Flag fields shouldn't read as raw numbers:

```
INSERT INTO types(name, kind) VALUES ('file_flags_e', 'enum');
UPDATE types SET is_bitmask = 1, size = 4 WHERE name = 'file_flags_e';

-- single-bit flags; omit the value and idasql auto-assigns 1, 2, 4, 8 ...
INSERT INTO types_enum_values(type_ordinal, value_name)
SELECT (SELECT ordinal FROM types WHERE name='file_flags_e'), v
FROM (VALUES ('FF_READ'),('FF_WRITE'),('FF_EXEC')) AS t(v);
```

Now **3** renders as **FF_READ | FF_WRITE** - in disassembly *and* pseudocode.

Strong abilities: structure offsets

The IDA "make this operand a struct field" power - as SQL:

```
-- show a raw [reg+8] access as cmd->payload
UPDATE instructions
SET operand0_format_spec = 'stroff:command_t'
WHERE address = 0x140001BE8;
```

Also: `enum:operations_e` · `sizeof:command_t` · nested `stroff:OUTER/INNER`

Raw offset math becomes readable field access.

Folders for everything

IDA's folder UI - funcs, types, names, imports, bookmarks, breakpoints - as a column:

```
INSERT INTO dirtree_folders(tree, path) VALUES ('funcs', 'review/annotated');
```

```
UPDATE funcs SET folder_path = 'review/annotated'  
WHERE address IN (0x140001BD0, 0x1400016A0);
```

```
UPDATE imports SET folder_path = 'imports/fileio'  
WHERE name IN ('fopen_s', 'fread', 'fclose');
```

Seven dirtrees, full CRUD - moves round-trip between headless and the live GUI.

One search across everything

```
-- find any named entity: function, label, type, member, segment
SELECT kind, name, printf('0x%X', address) AS addr
FROM grep WHERE pattern = 'cx_*';

-- byte-level: search, then patch
SELECT printf('0x%X', address) FROM byte_search WHERE pattern = '48 8B 05';
UPDATE bytes SET value = 0x90 WHERE ea = 0x140001C00;    -- patch to NOP
```

grep · byte_search · bytes (writable patches) · strings

And it keeps growing

Recent additions to the surface:

- **member-level xrefs** - `struct_member_xrefs` : who touches `command_t.op` , semantically (no disasm text scans)
- **fixed-layout type recovery** - freeze a struct, absorb gaps, no ordinal churn
- **bitmask enums** - flag fields render as OR'd constants
- **live byte patching** - `UPDATE bytes ...` , `DELETE` reverts, `is_patched` lists them
- **call-graph & shortest-path** - graph traversal as table-valued functions

Broad already - and still deepening. Same methodology across the family.

The CLI - and the prompt

```
idasql -s binary.exe -q "SELECT COUNT(*) FROM funcs" # one-shot  
idasql -s binary.exe -i                             # interactive REPL  
idasql -s binary.exe --http 8080                     # SQL server
```

A **raw PE** works directly - `ida lib` auto-analyzes on open. No pre-build.

`idasql/prompts/idasql_agent.md` - one monolithic markdown that teaches the whole surface to agents that don't support skills (analogue per tool).

Built the way you'll use them

The whole family was **vibe-coded** - agents building the tools that let agents reverse engineer. The pattern: **Claude Code** executes, **Codex** reviews and writes features & tests.

- **idasql** - the big one: **6+ months** of work
- **libghidra & ghidrasql** - almost entirely **Codex**; started just after idasql, **~a month**, then maintenance
- **bnsql** - same Claude-executes / Codex-reviews split
- **r2sql** - a **weekend** - **~5 days**, end to end

The xSQL family is the thesis in action.

PART 5

libghidra & ghidrasql

The hard member of the family

Why Ghidra is different

Engine	How the SQL tool reaches it
IDA	<code>ida.lib</code> - native C++ library, in-process
Binary Ninja	core API - native C++ library, in-process
Ghidra	Java / JVM - no native embedding API

ida.lib and BN drop straight into the C++ `libxsql` model.

You **cannot** just link Ghidra into a C++ process.

Three ways out

- rewrite ghidrasql in **Java**
 - ...which means embedding a SQL parser and re-implementing libxsql's virtual-table layer in Java - a second codebase
- **JNI** the whole application into C++ (heavy, fragile)
- put a **small server in front of Ghidra** and call it over RPC 

That bridge is `libghidra`.

Happy accident: going the service route is how we got `libghidra` + the RPC bridge as reusable artifacts.

libghidra - the bridge

A C++ RPC + offline decompiler bridge for Ghidra. One client, two backends:

```
client
├─ live    —▶ RPC (protobuf) —▶ LibGhidraHost (Java, real Ghidra)
└─ offline —▶ embedded Sleigh decompiler (no Java at all)
```

- **live** - talk to a running Ghidra, full project analysis
- **offline** - a decompiler path with no Ghidra UI and no JVM

A large surface you do not memorize

~88 RPCs across 9 services:

	RPCs		RPCs
Types	34	Memory	4
Listing	20	Symbols	4
Functions	15	Decompiler	2
Session	6	Health	2
Xrefs	1		

Enough surface for real work; agent- and binding-friendly by design.

The wire: protobuf over HTTP

POST /rpc on :18080

```
RpcRequest { method, payload }  
RpcResponse { success, payload, error }
```

Not gRPC - just a typed protobuf envelope. Pre-generated stubs ship with the repo.

One bridge, three languages

C++ · Python · Rust

Same protocol. Same model. Each binding ships a **live** quickstart and an **offline** quickstart.

Again: you don't learn a new API just to reverse.

Example - C++ (live and offline)

```
// live: talk to a running Ghidra host
auto client = ghidra::connect("http://127.0.0.1:18080");
auto funcs = client->ListFunctions(0, UINT64_MAX, 10, 0);
for (auto& f : funcs.value->functions)
    printf("0x%llx %s (%llu bytes)\n", f.entry_address, f.name.c_str(), f.size);

// offline: embedded Sleigh, no Java
auto local = ghidra::local({});
local->OpenProgram({.program_path = "binary.exe"});
auto d = local->GetDecompilation(0x140001000, 30000);
```

Example - Python & Rust

```
import libghidra
client = libghidra.connect("http://127.0.0.1:18080")
for f in client.list_functions(limit=10).functions:
    print(f"0x{f.entry_address:x} {f.name} ({f.size} bytes)")
```

```
let client = libghidra::connect("http://127.0.0.1:18080")?;
for f in &client.list_functions(0, u64::MAX, 10, 0)?.functions {
    println!("0x{:x} {} ({} bytes)", f.entry_address, f.name, f.size);
}
```

`pip install libghidra (+ [local] ) · crate libghidra (live + local features)`

How ghidrasql rides on libghidra

```
SELECT * FROM funcs
  → libxsql virtual table
    → LibGhidraSource          (ghidrasql)
      → libghidra client.ListFunctions()
        → RPC :18080 → LibGhidraHost → Ghidra → back
```

The SQL engine never knows it is talking to Java.

Two ways to run it

- **Managed** - ghidrasql owns the engine:

```
ghidrasql --ghidra <dist> --binary x.exe --http 8081
```

- launches Ghidra **headlessly (JVM)**, drives analysis
- on a **shutdown request**: cleanly **saves the project**, then terminates the headless Java process

- **Proxy** - attach to an already-running host (you own its lifecycle):

```
ghidrasql --url http://127.0.0.1:18080 --http 8081
```

```
:18080  ← libghidra ⇌ Ghidra    (protobuf)
:8081   ← you ⇌ ghidrasql      (SQL)
```

libghidra is a force multiplier

We built it so ghidrasql could join the SQL family - but on its own it's a full Ghidra automation surface, no GUI:

- **automation** - headless, scriptable batch analysis
- **quick prototyping** - try an idea against Ghidra in a few lines
- **agent-driven development** - let an agent drive Ghidra directly

C++ · Python · Rust · live or offline · agent-friendly

PART 6

AI integration

Who actually writes the SQL?

You do not learn the SQL

That is not the ask. Three layers make it usable:

- **developers** maintain the tables and the write semantics
- **skills** teach the agent the house style
- **the agent** writes the query it needs

Without skills: discovery. With skills: steering.

Without skills: the agent discovers the surface

It explores the database the way it would any unknown schema:

```
SELECT * FROM welcome;           -- one-row orientation: version, file, hints
SELECT name FROM sqlite_schema WHERE type='table';  -- what tables exist
.schema funcs                     -- (REPL) columns + types for a table
```

From there it knows the nouns and writes its own queries - **no prior knowledge of idasql.**

SQL is self-describing. The agent bootstraps itself.

With skills: steering

idasql-skills (and **ghidrasql-skills**, **bnsql-skills** - analogous):

connect	start · attach · route · verify · shut down
analysis	triage and prioritize risky areas
disassembly · decompiler	code · pseudocode · ctree · locals
xrefs · grep	callers/callees · find entities by name
types · data	structs/enums/prototypes · strings/bytes
annotations · storage	rename/comment/bookmark · durable notes

Same skill names, same SQL, three engines - that's the unification.

No skills? Ship an agent prompt

Not every agent supports skill packages. So each tool also ships **one monolithic markdown**:

```
idasql/prompts/idasql_agent.md  
ghidrasql/prompts/ghidrasql_agent.md  
bnsql/prompts/bnsql_agent.md      ( ...r2sql, pdbsql, clangsql, dwarfsql too )
```

It carries the whole contract - surface, routing, recipes - to drop into any agent's context.

Skills steer; the agent prompt is the portable, skills-less fallback.

One agent can hold all three (IDA · Ghidra · Binary Ninja)

- connect to all three in **one conversation**
- keep multiple binaries open at once
- compare the same function across engines
- copy names, prototypes, comments, or findings **from one database to another**

Cross-tool reversing becomes a conversation.

Large output is just a workflow

A giant function? Thousands of rows? The agent already uses the shell:

```
curl .../query > out.json    # then read it in chunks
```

Exactly like working with a large source tree.

Scale is handled with ordinary developer habits.

PART 7

Quick start

Two ways in: headless, or alongside your IDA

Install in five minutes

1. download a release - `idasql_cli-*.zip` (headless) and/or `idasql_plugin-*.zip` (GUI)
2. put `idasql` **next to the IDA runtime**, add it to `PATH`
3. drop the plugin into IDA's `plugins/` directory
4. verify: `idasql -h`
5. install the skills:

```
/plugin marketplace add allthingsida/idasql-skills
```

Same pattern for ghidrasql / bnsql.

Two modes - be clear which you want

	Headless	Assisted
ships as	<code>idasql_cli</code> (idalib)	<code>idasql_plugin</code> (in IDA)
who owns the DB	the agent	you + the agent, live
IDA GUI	none	open, in front of you
best for	triage, batch, scripting	visual verification, paired work

Headless: the agent runs IDA for you. Assisted: you share one live database.

Headless - just ask

```
/idasql:connect open C:/path/binary.exe in the background -  
                how many functions does it have?
```

Then route to the narrowest skill:

```
/idasql:decompiler    summarize the dispatcher  
/idasql:xrefs         who calls it?  
/idasql:annotations  comment the risky branches
```

No CLI to memorize - the connect skill drives it.

What `/connect` actually runs

Two pieces make headless work:

- **Background mode** — the agent launches `idasql` as a background process, waits for it to be ready, then reads `welcome` to orient itself
- **Server mode** (`--http`) — exposes the live database as a SQL-over-HTTP endpoint the agent queries with ordinary `curl` / `POST`

The agent already knows how to run, check, and stop a background server:

```
idasql -s binary.exe --http 8080          # start (background)
curl http://127.0.0.1:8080/status          # verify it's up
curl -X POST http://127.0.0.1:8080/query \
    -d "SELECT COUNT(*) FROM funcs"        # query over HTTP
curl -X POST http://127.0.0.1:8080/query \
    -d "SELECT save_database();"           # persist edits (or run with -w)
curl -X POST http://127.0.0.1:8080/shutdown # clean stop
```

Tell it in English. It manages the whole lifecycle.

Yes, there's `.mcp` too

For the MCP faithful - idasql speaks it natively (MCP over HTTP/SSE):

```
idasql -s binary.exe --mcp 9000      # MCP server (or .mcp start in the REPL)
.pin mcp 127.0.0.1 9000              # plugin auto-starts it when the DB opens
```

But the entire MCP surface is **one tool**: `idasql_query`.

MCP's transport - without the 117-tool sprawl. One tool, all the SQL.

Assisted - share your open IDA

In IDA, with the plugin loaded:

```
idasql> .http start          # expose THIS live database (or .pin http to auto-start)
```

Then hand the URL to the agent:

```
/idasql:connect work with this open database: http://127.0.0.1:<port>
```

Why `.http start` : your edits and the agent's edits land in the same live IDB - you watch it happen.

Many binaries, one conversation

```
/idasql:connect open every executable in this folder, keep each  
session running, and summarize function + string counts
```

- each binary gets its own live session
- query and **diff across all of them** in the same chat
- the cross-engine comparison from Part 3 - now routine

Saving & shutting down

- writes are live in the session, but **not persisted by default**
- persist with `SELECT save_database();` - or run the server with `-w`
- stop cleanly with `POST /shutdown`

```
/idasql:connect save intended changes and shut down all idasql sessions
```

The agent can save and tear down every xSQL family member it started.

PART 8

Vibe Reversing

Live

Some directions - live, with the SQL shown

The loop: **describe intent** → **agent writes SQL** → **the tool updates live**.

- **triage from scratch** - biggest funcs, imports, strings, the function everything routes through
- **annotate live** - rename, comment, and retype; edits land in the tool instantly
- **recover a type** - rebuild the tagged union + enum, apply them, re-read the pseudocode

Real-world results

The workflow from this talk - agent + SQL + RE tool - produced these:

Project	What	Tool
PoisonPlug	ScatterBrain C2 - 8 plugins, 3 ciphers, 337 functions	idasql
OceanDrift	Graph API / OneDrive C2 implant (APT28)	idasql
Handle64	PROCEXP152.sys kernel driver audit - 16 IOCTLs	ghidrasql
10 BYOVD drivers	typed client libraries, no WDK needed	idasql

github.com/Oxeb/vibe-re

PoisonPlug / ScatterBrain

A modular C2 framework - 4-stage loader, 8 encrypted plugins, 337 functions.

- **3 cipher systems** reverse-engineered: rolling XOR, polynomial XOR, IMUL stream cipher
- **8 plugins** extracted and rebuilt as valid PE DLLs (Install, Plugins, Config, Online, TCP, HTTP, UDP, DNS tunnel)
- **6-stage automation pipeline** - extract → rebuild PE → decrypt blobs → reconstruct plugins → decrypt strings → annotate via idasql
- **zero sub_* remaining** - all 337 functions fully named

The annotation stage runs idasql as an HTTP server per plugin (ports 8200-8207), batch-renaming globals and locals via SQL.

OceanDrift (Graphite / APT28)

A 32-bit implant that uses Microsoft Graph API + OneDrive as a dead-drop C2.

- **OAuth2 token refresh**, OneDrive folder staging (/job , /result , /upload)
- **7 commands**: kill, shell, exec, upload, download, sleep, rest
- **config crypto**: hex-encoded + 16-byte cyclic XOR, ENCR: marker
- **host fingerprint**: MAC + CPU ID + disk serial → MD5 → machine GUID
- **full C++20 reconstruction** - compilable source from idasql decompilation
- **YARA rules + MITRE ATT&CK mapping** included

Entire analysis done SQL-only - no IDAPython, no GUI clicking.

Handle64 / PROCEXP152.sys (ghidrasql)

Security audit of the Process Explorer kernel driver - Microsoft-signed.

- **16 IOCTLs** fully documented - two are exploitable read primitives
- **PPL bypass:** `PsLookupProcessByProcessId` skips protection checks - enumerated all 1,969 handles in `lsass.exe`
- **full ntoskrnl dump:** IOCTL 0x83350044 reads the live kernel image (~16 MB), defeating KASLR
- **1,106 lines of reconstructed driver source** - compilable, no WDK needed
- **live vs. on-disk diff** discovered boot-time retpoline patching (Spectre v2)

Analyzed headless on two concurrent ghidrasql instances (ports 8081, 8082).

BYOVD: 10 vulnerable signed drivers

Client libraries for 10 signed Windows kernel drivers - no WDK required to build.

Driver	Key capabilities
DNDrv (VBoxDrv)	ring-0 code loading, kernel page mapping, 30+ IOCTLs
GGProtect64	60+ IOCTLs: DLL injection, SSDT, callback stripping
KmWpsMs (NcHost)	arbitrary physical memory R/W, VA-to-PA translation
bin_intigua	IAT hooking of services.exe, shellcode injection, PPL bypass

+ Cormem · mst · gibepext · SysFile_X64 · fastdumpx64 · ImmUNETUtilDriver

The workflow: ask the agent to re-source the driver via ghidrasql - recover IOCTL dispatch, structures, and constants - then write a typed client library and demo program. From binary to usable SDK in one conversation.

[Video walkthrough](#)

Recap

- an analyst asks ordinary questions
- **libxsql** makes the evidence queryable - and writable - live
- **idasql** exposes the whole IDA model as tables, views, and functions
- the same idea spans IDA, Ghidra, and Binary Ninja
- **libghidra** brings the Java engine into the C++ SQL family
- **skills** let the agent drive, so humans don't memorize schemas

LOOKING AHEAD

Where this is going

- **one unified SQL interface** for every RE tool – still **WIP** (idasql · ghidrasql · bnsql · r2sql; more to come)
- **collaboration** – `idasql-server` + family server components: shared live sessions for teams
- **grow by table, not by tool** – new capability = a column / view / function, never the 201st tool
- **token efficiency** – spend context on the binary, not the machinery
- **local-inference first** – less reliance on frontier models

One mental model · one surface · any engine · any agent – local included.

Get the tools - all open source

SQL interfaces

idasql
ghidrasql
bnsql
r2sql

Foundation

libxsql
libghidra

Skills - Claude & Codex

idasql-skills
ghidrasql-skills
bnsql-skills
r2sql-skills

@allthingsIDA · YouTube

The binary is a database.

Go `SELECT * FROM` it. |